Department
of Information Engineering and Computer Science

Faculty of Informatics

Master's Degree in

Data Science

FINAL DISSERTATION

Multilingual Address Extraction from Websites

[redacted]

Andrea Passerini
UniTrento Supervisor

Máté András Cserép
ELTE Supervisor

[redacted]

Gábor Vitrai, MAT. 230360
Student

ACADEMIC YEAR 2021/2022

Acknowledgement

I would like to acknowledge and express my sincere thanks to my industrial supervisor, Alessio Guerrieri, for offering me this topic and internship. I want to thank my mentor in SpazioDati, Stefano Frasetto, for playing a critical role in the completion of the thesis. I wish to express my sincere thanks for helping me understand the new tools and giving me advice on how to progress on a daily basis.
Special thanks to both of my university professors, Andrea Passerini and Máté Cserép András, for being committed to being my academic supervisors.
Many thanks to my family for supporting me throughout my studies, even abroad.

Contents

Chapter 1

Introduction

1.1 The task

The problem this thesis aims to solve can be described with the following example: having the textual data of a web page/website, find and extract the postal addresses from the text in any given language. Target languages include English, Italian, German, Norwegian, Spanish, French, Dutch, Greek. To clarify a common misconception about information extraction regarding addresses, let us see the difference between Address Parsing and Address Extraction.

1.1.1 Address Extraction

The thesis seeks to discuss possible solutions for address extraction on multilingual data, provide the tools to create a balanced dataset that can be used to train a multilingual model for address extraction, and create a prototype that can take textual content as an input, returning the postal addresses from the text. These tools, methods, and approaches are described in this thesis in detail.

Input:
Re-think and improve the data management lifecycle in your organization. Quickly prototype large scale data integration projects. Implement Open Data strategies and campaigns. Design, integrate and apply ontologies. Leverage the hidden value in your data. [..] privacy policy - cookie policy © 2013 - 2022 SpazioDati S.r.l. – **Via Adriano Olivetti 13, 38122 Trento** (Trento) – REA TN 210089 – Tax code and VAT ID 02241890223 – Group VAT ID 12022630961

Output:
Via Adriano Olivetti 13, 38122 Trento

Figure 1.1: Example of Address Extraction

1.1.2 Address Parsing

Address parsing is the process of classifying each address component of a known, full or partial address. These components can be street name, street number, zip code, country name, country code and many others.

Input:
Via Adriano Olivetti 13, 38122 Trento (TN)

Output:

```
}
      "house_number": "13",
      "road": "Via Adriano Olivetti",
      "city": "Trento",
      "state": “TN"
      "postcode": "38122"
}
```


Figure 1.2: Example of Address Parsing

A popular and advanced tool for address parsing is libpostal [Openvenues, 2018], which is equipped with an address extension feature as well. The package, for example, is capable of expanding partial addresses, and it can also resolve address abbreviations, like *Via G., 3, 40124 BO*, which stands for *Via Garibaldi, 3, 40124 Bologna*. The software was not used to develop the thesis, and it only serves as an example use case for address parsing. The task of normalizing and parsing addresses does not appear in the list of set goals for the thesis.

The main challenge for the project is to create a trustworthy solution for multilingual usage, not only for a single language. This requirement aspect makes it challenging, mainly because postal addresses differ significantly from country to country and from language to language. Having a single language opens possibilities for different non-machine learning solutions; however, the question this project also aims to answer is to see if multilingual models got advanced enough to replace the old-school regex-based or older ML solutions created to extract data given a single language.

1.1.3 Internship

This thesis was submitted in fulfillment of the requirements, for the double degree of Master of Computer Science, as an output of the compulsory internship in collaboration with SpazioDati Srl. *(Trento, Italy)*. The thesis was submitted to the **Eötvös Loránd University** and the **University of Trento** in two separate copies.

1.2 The planned solution

1.2.1 Structure of the Solution

The introduced solution follows the structure shown below, consisting of the following steps:

1. Pre-processing
    (a) Tokenization
    (b) Labeling
    (c) Chunking (Sequence generation)
2. Dataset Generation
3. ML Model Training
4. Address Extractor Pipeline
    (a) Model Prediction
    (b) Address extraction with Regex

1.2.2 The output

The main topic of the thesis is to conduct extensive research while discussing the feasibility of a Machine Learning approach. The dataset, which belongs to the company will not be submitted; however, the tools to recreate the project shall be available.
The output contains the Package for data pre-processing, Tool for dataset creation, the Jupyter notebook for model training, and the code for the pipeline which was used for the final information extraction.

Chapter 2

Problem Definition

2.1 Motivation

SpazioDati creates and sells an extensive database of companies with information coming from official and unofficial sources. In the future, they would like to extend their services to the whole EU, so the company is crawling websites across Europe, and they need to analyze the data they have collected.

The company already has the physical addresses of millions of companies. On the other side, millions of crawled websites are given. Having these separated information sources, the company needs a solution to connect them with high precision.

2.2 Aim of the project

SpazioDati's overall goal is to match the addresses of the companies with the corresponding websites. In order to attempt linking a given company to a random website, first they need a solution to find the address of the company in the text of the corresponding website. Once the address was found in the text, the address can be used to link it with the other database, where the address of companies are known, but the websites are not. For this process, other simple attributes like the name of the company might be used, however linking with addresses is especially useful, since a company might have multiple shops/offices listed under their "Contacts" page.

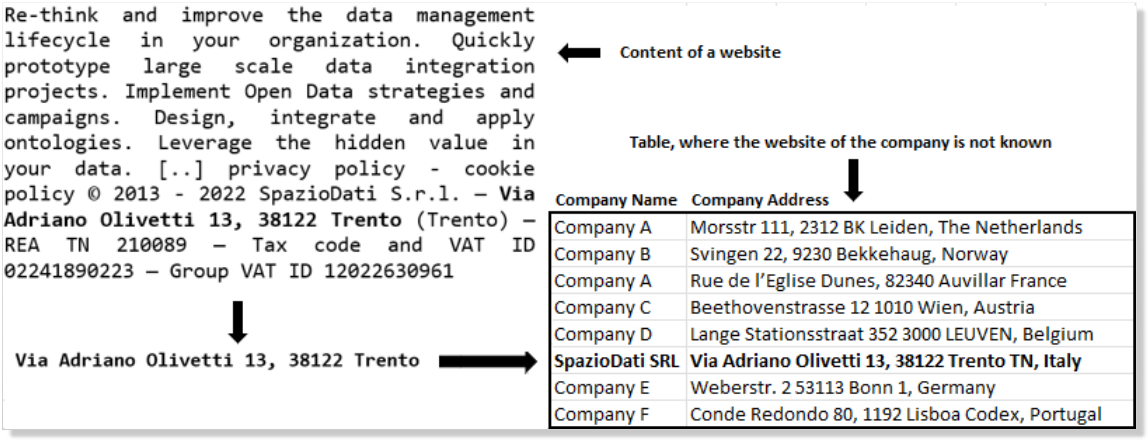


Figure 2.1: Linking a company to a website

The aims of the project can be listed in the following points:

- Do research on what is the state-of-the-art solution for this problem field
- Answer the proposed scientific questions
- Discuss possible solutions and see how feasible are they
- Conduct Experiments
- Create a prototype
- Evaluate the prototype
- Make suggestions for possible developments and improvements

2.3 Resources

2.3.1 Dataset

The biggest challenge in creating a multilingual solution is getting the data. For a Machine Learning approach, a massive corpus of organic, real-life data is needed to train a model in a balanced way. During the project, synthetic data is also introduced, which means that known addresses are inserted into the text. If partial addresses were found in the text during the first step of pre-processing, addresses are inserted into a part of the text, where it could occur in real life.

Such data sets are not available online to the public, which is one of the main reasons multilingual address extraction is not available online. Since SpazioDati is actively crawling, collecting, and buying data of companies and websites, a sufficient amount of data was available for the project. From a thesis view of point, it is important to note, that due to lack of data, Hungarian is not a target language. A brief overview of the dataset that the company provided can be see below.

Table 2.1: Provided web pages / language

Language	Count	Percentage
English	681 674 web pages	24,7%
Italian	145 421 web pages	5,26%
German	228 828 web pages	8,27%
Norwegian	235 001 web pages	8,49%
Spanish	1 206 583 web pages	43,63%
French	53 993 web pages	1,95%
Dutch	211 324 web pages	7,63%
Greek	2 110 web pages	0,07%
TOTAL	2 764 934 web pages	100%

2.3.2 Hugging Face

The Hugging Face transformer package and its APIs were used for development, which provides pre-trained, valuable models for many given NLP tasks. Pre-training is usually done on a vast amount of data, which takes an immense amount of processing power. On the other hand, fine-tuning is training a model after being pre-trained, which is a massive advantage since only a single classifier layer needs to be trained; plus, this fine-tuning requires much fewer data to get good results. This way, training also takes less time and fewer resources.

Chapter 3

State-of-the-art

3.1 Research Questions to be answered

The thesis aims to answer a few theoretical questions:

1. What is the state-of-the-art solution for multilingual NLP?
2. How can we use a large set of example addresses to build an Addresses Detection Model?
3. How can we make this process efficient for millions of websites?
4. How will the quality differ in different European Countries? Will the alphabet influence the results? (In terms of the quality/precision)

3.2 Research, methodologies

This section presents an overview of the initial research, including techniques and methodologies that were used as guidance for creating the whole pipeline, including pre-processing, Machine Learning techniques, and address extraction techniques. Some examples will show earlier solutions of slightly different extractors or parsers, listing their purpose, weaknesses and problems. Most importantly, this section will also talk about the new, State-of-the-art solution for this problem field in 2022.

3.2.1 Literature and background of the problem area

English Postal Address Extractor

Extracting an address is not a new problem field. 15 years ago, the tools and technology was not given, in order to create a multilingual address extractor, the technology only allowed researchers to create solutions for single languages. The main issue was the lack

of processing power to train Machine Learning models, and the fact that the models were not sophisticated enough to handle multiple languages on a given problem field. A similar project was also done in 2007 [Yu, 2007], where the goal was to create a high accuracy address extractor for English only. The mentioned work details using different approaches targeted for the single language, and it even details some results regarding the performance of combining them. For instance, the first approach is a Regex-based solution (Regular expressions), which is feasible for a single country where the address format is similar. This thesis aims to explore possible solutions to create a faster prototype, possibly with Machine Learning, since having multiple complicated Regular Expressions are difficult to create and maintain. In the case of the High Accuracy Postal Extractor [Yu, 2007], the author also talks about a Machine Learning approach. The author was using Conditional Random Fields (CRF) since, by that time, that was the most used technique for NER tasks. NER stands for Named Entity Recognition, which is a sub-field of Token Classification. CRFs are undirected graphical models. This model type defines the conditional likelihood of labels given a particular observation in the given sequence, which means that the model is trying to decide how a list of tokens (sequence) should be labeled, using other components in the same sequence.

DeepParser

Deep Parser [Yassine et al., 2020] is a recent work done by a group of researchers. The project [Abid et al., 2018] is open-source, and it details a solution for a Trainable Postal Address Parser. As discussed at the beginning of the first chapter, parsing is a different problem, and this thesis does not aim to provide a solution for that. This paper is mentioned as it discusses a trainable solution, which means that in case a dataset of postal addresses is given, anyone can train their custom model. Their solution takes an image or text of an address, and the tokenized address is injected into a classifier in 3 different ways: words, character trigrams, and characters.

Map Maker

The last project that was found interesting to mention, is working on address extraction of US addresses, was MapMaker [Chang and Li, 2010] which was published earlier in 2010. The authors *Chia-Hui Chang* and *Shu-Ying Li* proposed a similar approach to the problem, using **CRF and SVM for sequence labeling**. Pattern-based address extraction approach was utilized, which used several address patterns and a small gazetteer (a gazetteer is a geographical dictionary). Elimination of HTML tags was required so the

model would not learn unnecessary words.

3.2.2 Multilingual Models

In the last 3-4 years, ML models have been published by the most prominent research groups, including Google, Microsoft, and Facebook. Google released the famous BERT and mBERT models, next to their T5 and mT5 models. While Google and Microsoft's models are tailored for NLP tasks like question answering, Facebook has concentrated on the topic of neural machine translation. Unfortunately, Microsoft's models are not open-source, but Google's and Facebook's models are publicly available on GitHub [BERT, 2022]. During testing, mBERT proved to be the best (detailed in **Chapter 6. Experiments**), and agreeing with SpazioDati, different variants of this model started to be used (*Base Bert Multilingual Cased* and *Base Bert Multilingual Uncased* provided by Hugging Face).

3.2.3 Conclusions

Concluding from all the resources, it can be stated that so far, such address extractors have only been made for English or that such projects can not be found openly on the internet. The main issue with the creation of such projects is the collection and preparation of the data. As mentioned above, new, State-of-the-art models are available, and experiments/pioneering developments are made using these models. However, such a multilingual address extractor is not publicly available based on our research.
Earlier solutions are only working on one language, using old models like CRF or SVM, while utilizing power-demanding solutions like regex and pattern mining. Each extractor project uses some sort of BIO or BILOU labeling. In the following points, solutions, tools, and techniques will be discussed that are required to reach a decent performance for the development of a multilingual solution.

- Multilingual models for Address Extraction
  - **Multilingual** models got to a fairly advanced level
  - **Bidirectional** Encoder Representations from Transformers
  - The mBERT model is pre-trained with more than a **100 languages**
  - It is sufficient to **fine-tune** it on a subset of the languages
- Sequence labeling and n-grams
  - Address Extraction is a type of sub-field of **Token Classification**
  - Token classifiers are usually trained on **sequences**

  - Generating **n-grams** from web pages will be required

- Balanced Dataset
  - The dataset is crucial for the training of the model.
  - Each language needs to be **balanced**.
  - Some of the sequences need to **contain an address** some of them do not.

- Extraction Pipeline
  - The tokenized output of the model needs to be **analyzed**
  - The decision needs to be made, regarding what sub-sequences to consider as an **"accepted address"**

3.2.4 Problem Field: Token Classification

The generic task of token classification encompasses problems formulated as "assigning a label to each token in a sentence." This topic can be divided into many categories, but the three most relevant ones for the current topic are the following:

Named Entity Recognition (NER)

NER [TowardsDataScienceNER, 2022] is used to find entities in a sentence. Such as *people* or *locations*. In the case of Named Entity Recognition, each token can have one class or can be labeled as not an entity.

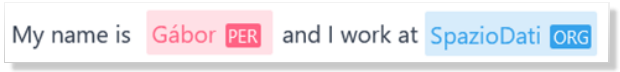


Figure 3.1: Example of Named Entity Recognition

Part-of-Speech tagging (POS)

POS [TowardsDataScience, 2022] is very similar, here each word in a sentence can be labeled to a specific part of speech such as a *verb* or *adjective*. This type of entity recognition can not be used for addresses.

Figure 3.2: Example of Part-of-Speech Tagging

Chunking

In case the target is to extract only one type of entity, (address) chunking is the most suitable solution [D'Souza, 2022]. Chunking is an extension of NER that takes NER tags as input and outputs chunks. Generally, chunking can be done with different entities at the same time. Entities can be labeled as addresses (B-ADD, I-ADD, O-ADD) and as products simultaneously (B-PRO, I-PRO, O-PRO). For the set task, only address-type entities are going to be labeled.
The required labels are the following: BEGINNING (B) of a chunk, INSIDE (I), marking components that build the inner parts of the address (or any entity in general). The last type of label is OTHER (O), which gets applied to every other non-chunk component. **Please note that chunking is similar to NER, however, here, a single entity can consist of multiple tokens, marking the "borders" of the entity.**

Another point that needs to be mentioned is that different labeling methods are available, and each solution provides its variant. With the so-called BIO tagging method, the ending token is not marked separately. The figure below shows the BILOU variant. There is a slight difference between the two labeling methods, which will be detailed in the **4.2.1. Labelling Methods** section.

Figure 3.3: Example of Chunking with BILOU tokenization

Chapter 4

Dataset Pre-processing Pipeline

Dataset creation is usually the most delicate and time-consuming part of large-scale Machine Learning projects. The dataset must be collected, prepared, pre-processed, balanced, sampled, shuffled, and split. The pre-processing pipeline was designed with many configurable options and features to have everything done easily.
Note: As a result of the input data structure, for each web page, only one known address is given for each company. Luckily, since the model will be trained on sequences of tokens and not on full web pages, this will not be misleading for the model. Nevertheless, it must be kept in mind that **a web page can contain multiple postal addresses, including full or partial addresses**.
Since the project requires work on big data, the procedures had to be well optimized, and I/O operations had to be well implemented.

The data is read in chunks since it is essential to avoid reading many gigabytes of data at once. The pre-processing pipeline consists of 4 significant steps: Tokenization, Labeling, Sequence Generation, and Dataset splitting. These steps are divided into different python scripts where the first three are called by the *init.py* script for every chunk of data. The last step, dataset splitting, was designed to be a separate process since the user might want to generate different subsets/splits from the result of the pre-processing.
The data is passed between the scripts after each script has finished processing the given chunk of data. Each scrip has its own variant of data structure, but the returned value is always a Data Frame.

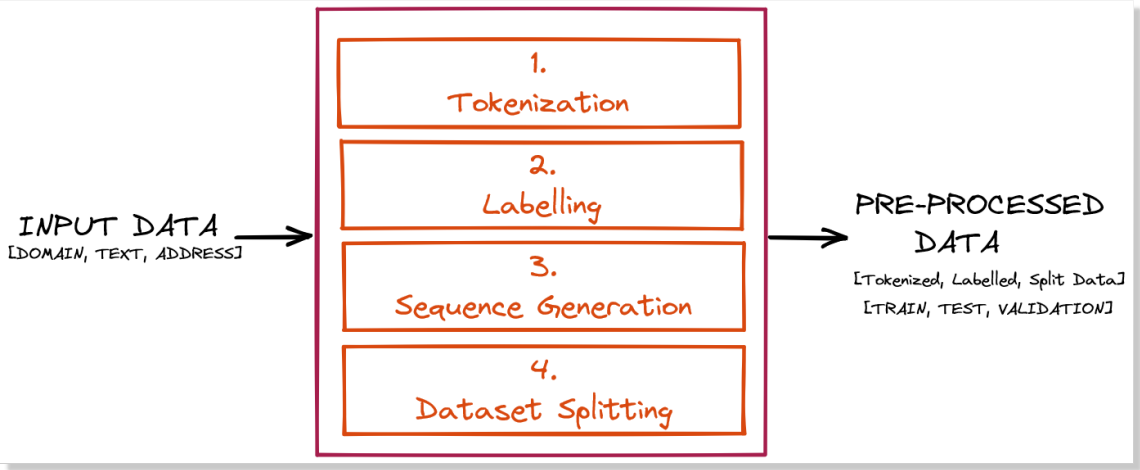


Figure 4.1: The Components of the Pre-processing Pipeline

The input provided by the SpazioDati was prepared in JSONL files, each containing JSON objects. Each record contains a given web page's data. Using the *pandas* package, the *pandas.read_json()* method is called. It converts a JSON file to a pandas object, using the optional parameter "lines" to read a given file with JSONL structure. Since the result of the pre-processing has to be sequenced, it is not necessary to merge all the pages/sub-pages of a website into a single record. Having the pages in separate records results in shorter parts of texts, making processing faster. A given record looks like the following:

```
{
    "domain":"unitn.it",
    "content":"...",
    "lang":"it",
    "address":"Via Calepina, 14, 38122 Trento TN, Italy",
    "source":"source0"
}
```

A given **domain** can occur multiple times in the dataset, as there is no need to have the full URL of a page (please note that a website can consist of multiple web pages). The **content** field contains the pure textual content of the page excluding HTML tags or other programming-related scripts/metadata (the content of the header and footer is kept as the footer contains addresses in many cases). The language of the **page** was determined by a different model used by SpazioDati. This attribute is important as the project's main advantage is multilingual data, and a domain may contain pages in multiple languages despite the *ccTLD* (Country-code top-level domain) of the website (e.g.: *unitn.it* and *unitn.it/en*).

Given the example of the University of Trento, since multiple departments belong to the institution and each of them may have different postal addresses, it results in a single page having different addresses in its content. In this case, despite the content containing multiple addresses, the given record will be used to train only **one**, known address. Other addresses will be trained using other, separate records from the input dataset. In conclusion, upon evaluation of the Machine Learning model, it must be considered that real but not "known" addresses might be present in some of the text used for training. This property even allows using a text/webpage used for training to check if the model can find other addresses in the exact text, aside from the one it was trained on for that specific web page (this usually can not be done, however, the particular type of dataset makes this type of manual testing possible).
Before running the *init* script, multiple parameters can be set. The white-listed languages that need to be processed or the size of a chunk that defines how many website objects will be sent through the pipeline that will be computed simultaneously can be set here.

4.1 Tokenization

First, the tokenizer script is called. In the beginning, the text and the known address are set to lower case, and punctuation and special characters are removed. To determine if the known address can be found in the text or not, the program checks what amount of the address components can be found in the text where *street name*, *ZIP code*, *country name* or related components are considered as tokens. The address is said to be inside the text if at least three components could be found in the text. This solution might seem like a strange way to decide; however, upon working with addresses, especially with multiple countries in the dataset, the structure of the address that needs to be found might be different from the one that is attempted to be found (based on the known address). Not finding the known address in the text (referring to as the address which came with the dataset) does not mean that there are no valid addresses in the text; however, later on, upon creating synthetic data, the script needs to handle these kinds of records differently. As a result, the location(s) of the found address will be saved to the output, with other details included. The generated output of the tokenizer is a Data Frame with the domain, the originally known address, a Boolean marking if the address is considered found in the text, the extracted address, the text of the page, plus an extra "*details*" dictionary object containing extra relevant information. This information includes the website's language, the set of components of the original and found addresses, their locations, and lengths in the text. These stored data are used for measurements later on.

Output format: Data Frame

```
URL                                                          "unitn.it"
address                          "Via Calepina, 14, 38122 Trento TN, Italy"
extracted_address                             "Via Calepina 14 38122 Trento"
is_address_in_text                                                    True
text                                                  "internazionale..."
details
{
'original':  {'14', '38122', 'Calepina', 'Italy', 'TN', 'Trento', 'Via'},
'found':                     {'14', '38122', 'Calepina', 'Trento', 'Via'},
'lang':                                                              'it',
'found_address_locations':                                     [6584, 28]
}
```

4.2 Labeler

Labeling is the process of assigning a label to every token inside a sequence/text. In the cases of websites, tokens are single words split from the text, crawled from the website. This process excludes HTML tags and Metadata.

4.2.1 Labeling methods

Before diving into the features of the Labeler, let us discuss different labeling methods.

BIO labeling

BIO stands for Beginning (B), Inside (I), and Outside (O), representing if a segment of a text/sequence is part of the information that needs to be found. This kind of tagging can only be used to recognize boundaries. [Ramshaw and Marcus, 1995]

BILOU labeling

BILOU tagging is similar, but it uses more accurate solutions for chunking. This type of encoding marks the Beginning, the Inside, and the Last token, with the addition of Outside and Unit-length tokens. For some specific data sets, BILOU has outperformed the more widely used BIO.
Both of the above-mentioned labeling methods can be used to extract the different types of entities using Token Classification; however, this project aims not to parse an address but

to extract one single type of entity: an address. As a result, there is no need to introduce different types of tokens like B-ZIP Code, B-Street Name, I-ZIP Code, or I-Street Name as BILOU encoding would do. As mentioned earlier, only address types need to be labeled.

4.2.2 Used labeling - BILCOU

Papers and evaluation libraries discussing the topic of address extraction often used some variant of BILOU labeling, as for a given language, the addresses usually have a similar structure. For example:
Italy: <Additional Information (floor / door)> <Street name> <Street number> <Postal code> <City> <Province or Province abbreviation> <Country>
Hungary: <Country> <City> <Street name> <Street number> <Additional Information (floor/ door)> <Postal code>

While the mentioned paperwork used a variant that could be abbreviated as BILO (no use of UNIT length tokens). To give more flexibility to the extractor work the following tags were introduced:

- S = Start (Beginning of the address)
- M = Middle (Inside component of an address)
- E = End (Last token of the address)
- C = Component (Possible component of an address)
- O = Other (Tokens which are not part of an address)

These label names were applied through pre-processing; however, later on, for the training of the ML Model, these label names are going to be renamed to the conventional BILOU encoding, where "B", "I", and "L" are going to be part of the ADD (address) entity, and the Component labels will be using the UNIT labels. (B-ADD, I-ADD, L-ADD, U-COM)

Component Tokens

Component tokens can occur anywhere in the content of a web page. This token was introduced to help fill the gaps in extracted addresses. Finding an address in the text is difficult since the known address might appear slightly different in the text. Thus, it must be considered that there is no guarantee that the known address can be found in the text.

Furthermore, data analysis highlighted that in the majority of the cases, the address which can be found in the text is usually only a partial address.

a) These words might belong to the full address but are not recognized as a part (e.g.: Country Name / Country Code). Upon examining the dataset, it was noticed that the country name or the country code is not part of the address in many cases, but in reality, it often appears in the text as part of the address. Adding the country names in different languages to the dataset helps the model recognize if these types of tokens belong to the address or not.

```
Output: ('Hohentwielsteig', 'S'), ('6', 'M'),
('14163', 'M'),  ('Berlin', 'E'),  ('Germany', 'C')
Corrected: ('Hohentwielsteig', 'S'), ('6', 'M'),
('14163', 'M'),  ('Berlin', 'M'),  ('Germany', 'E')
```

b) Upon address extraction, COMPONENT labels can help us extract poorly recognized addresses. There might be cases where the full address is labeled as two small addresses due to the structure of the address. This case can be the result of faulty labeling. However this scenario, the component labels can be used to merge the extracted components.

```
Output: ('Via', 'S'), ('della', 'M'),  ('malpensada', 'E'),
('90', 'C'),  ('38123', 'S'),  ('Italy', 'M'),  ('Trento', 'E')
Corrected: ('Via', 'S'), ('della', 'M'),  ('malpensada', 'M'),
('90', 'M'),  ('38123', 'M'),  ('Italy', 'M'),  ('Trento', 'E')
```

c) Using the COMPONENT tokens, the script can make improvements to increase the accuracy of **START** or **END** labels. A half-correct classification can be seen in the example use case below. In this example, the ZIP code did not get labeled correctly. Budapest was selected as a Component rather than the beginning of the address. In a case like that, the extractor can return the last four tokens as an output instead of only 3 ("Budapest, Almafa Utca 55" instead of "Almafa Utca 55" only).

```
Output: ('1121', 'O'), ('Budapest', 'C'), ('Almafa', 'S'),
('utca', 'M'),  ('55', 'E')
Corrected: ('1121', 'O'), ('Budapest', 'S'), ('Almafa', 'M'),
('utca', 'M'),  ('55', 'E')
```

Please note, that this example serves as an example and Hungarian is not a target language.

What was used to label tokens as components: Component labels were chosen based on different conditions:
a) The set of tokens of the known address (e.g. if the address has a token "via" or "Trento," then all the occurrences of these words will be selected as components after the other standard (S, M, E) labels were selected).
b) Country names were also introduced in a language-dependent way. A collection of country names were utilized from an open-source dataset [Stefangabos, 2022]. This list was modified to have all the country names for each language (e.g., In **Italian**, Italy is called Italia, and Hungary is called Ungeria. Similarly, in **Hungarian**, Italy is called: Olaszország, and Hungary is called Magyarország).
c) A third addition is a long list of address components listed separately for each language. Address components can be different types of street names and level names (e.g. in English: "floor", "fl", "level", "lvl", "basement"). These include abbreviations, which are necessary to resolve as they are often written differently in text and in the case of an official postal address (**please note** that abbreviation expansion/resolving is not part of the project, the only aim is to classify these tokens correctly). This dataset was copied from the Libpostal's GitHub repository [Openvenues, 2018].
By default, next to the language of the given page, *English* is also forced on the text, as in many cases, English components can occur in many web pages in different languages.

Expectations

As a multilingual model is used, it should be able to recognize the different address structures inside a text. Furthermore, it should be able to handle a foreign address in a different language environment (e.g., a German address in an English text). Our expectation is that START and END tokens will be the most difficult to classify while MIDDLE and COMPONENT will have a higher shore upon classification.

The reason for this is simple: Each country has its format of an address, while there are multiple well-known, often occurring components as well. As an example, in the case of Italy, many addresses start with "Via", which should be easy to classify. Nevertheless, there are other tokens/words an Italian address can start. Based on the expectations, in the case of different keywords, like "street" in distinct languages: La via, Straße, and Utca tokens should be recognized by the model with higher precision.

4.2.3 Configuration of the Labeler

Since labeling had to have multiple features, the primary method of the script was made to be configurable. The user can decide if he/she wants to use standard or component labels separately. As a next step, the user can decide if he/she wants to use the above-mentioned additional list of address components and country names. The lists are used by default to increase the number of words the model has seen before.
The option to generate synthetic data is also given in the script. The user can determine what percentage of the total dataset he wants to modify into synthetic data. The generation of such data follows the following procedure:
First, all the known address tokens (words that were part of the initially known address) will be removed from the text. This removal feature is separately also available for text cleaning purposes as an option; however, the synthetic generation runs the cleaning by itself, as it is a mandatory requirement. Next, in case a partial or complete original address was found, the known address is inserted into its original position; otherwise, it is inserted randomly between two words.
Since synthetic data might contain addresses in a random position of a sentence, without context, these pages are marked as "synthetic" record types.

4.2.4 Address Patching

The last optional feature is address patching. There are cases when an address inside the text contains more information than the initially known address in the dataset. This situation can cause the model to learn valid address components as they would not be elements of the address, resulting in an OTHER label instead of the token being part of the extracted address. Enabling this feature will result in a maximum of 1 extra token being assigned to the address if the script encounters such a token. Enabling more than one token would risk appending unrelated words to the address.

Known address	In the text
Grete Nevermann Weg 14 22559, Hamburg	Grete Nevermann Weg 14 **d** 22559, Hamburg
Theodor Althoff 1 45133, Essen	Theodor Althoff **strasse** 1 45133, Essen
Pont West 107 9600, Ronse	**Nás** Pont West 107 9600, Ronse
Organki 2 31 990, Krakow	**Techramps** Organki 2 31 990, Krakow
Hoyo 29620, Torremolinos	Hoyo **28** 29620, Torremolinos

Figure 4.2: Patched Addresses

Thanks to patching, in the case of the following examples (figure 4.3 and 4.4), the miss-classified four tokens will also create a complete address during extraction.

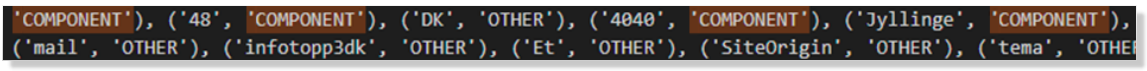


Figure 4.3: Example 1 of Address Patching

Figure 4.4: Example 2 of Address Patching

In these examples, the model would mainly learn how COMPONENT tokens look however, after address patching, they would look like the following. In example 1, the token "DK" will be part of the extracted address. In example 2, the token "DK" will be patched and as a result, the partial address on the left will be merged with the component tokens on the right, forming a full address. Resulting in this format: "('...', 'START'), ('...', 'MIDDLE'), ('...', 'MIDDLE'), ('...', 'MIDDLE'), ('...', 'END')"
The Output format is a single Data Frame, containing a record per web page with the following fields:

```
url                                                    "unitn.it"
known_address                 "Via Calepina, 14, 38122 Trento TN, Italy"
extracted_address                          "Via Calepina 14 38122 Trento"
labelled_text                               "('internazionale', 'O')..."
lang                                                          'it',
synthetic                                                     False
```

4.3 Chunk Generation

Since Token Classifiers work the best on small chunks of data (so-called sequences), each web page needs to be split into N long sequences. The procedure is simple, the only criteria being that an address can not be split into two sequences.

Figure 4.5: Example for Ideal Splitting with record types

Aside from the generated sequences, the numerically **encoded** labels are also added to the given record of the chunk. These labels are needed as Machine Learning models need the numeric equivalent of the labels. This Encoding turns the labels into integers in the following way:

START = 1 | MIDDLE = 2 | END = 3 | COMPONENT = 4 | OTHER = 0

The ID of OTHER (O), means the word does not correspond to any entity. For statistical purposes, chunks got categorized based on record type. In the project, three distinguished types of sequences were introduced.

1. **SME**: Stands for **Start, Middle, End**. These sequences contain a full address. After pre-processing, a website page gets split into sequences (chunking). Among these sequences, only one will become an SME record since, for each website, only one address is assigned to the company in the dataset.

2. **C**: Stands for **Component**. These records do not contain addresses; however, they have at least one token labeled as Component.

3. **O**: Stands for **Other**. All the remaining records are categorized as Other records (strictly consisting of Other labels).

Including other relevant details like language and a variable marking whether a sequence is organic or synthetic, the data is exported to a single **Apache Parquet** [Vohra, 2016] file. *Apache Parquet is an efficient, structured, column-oriented, compressed, binary file format.* Originally the output of the pipeline was selected to be a JSON file; however, the output turned out to be unreasonably big. A 10 GB input file (web page - address pairs) was exported as a 40GB JSON file, while the same input becomes 4 GB using Parquet.
Output format: Apache Parquet file

```
content          ['Università', 'di', 'Trento', 'via', 'Calepina ,...]
address                    "Via Calepina, 14, 38122 Trento TN, Italy"
labels                              ['O', 'O', 'C', 'S', 'M', ...]
label_ids                           ['0', '0', '4', '1', '2', ...]
lang                                                          'it',
synthetic                                                     False
record_type                                                   "SME"
```

4.3.1 Pre-processing Recommendation

In the case of using the pre-processing pipeline, it is recommended to run the pipeline separately on each language. This way, the user can have a separate parquet file for each language, which helps to create a balanced dataset.

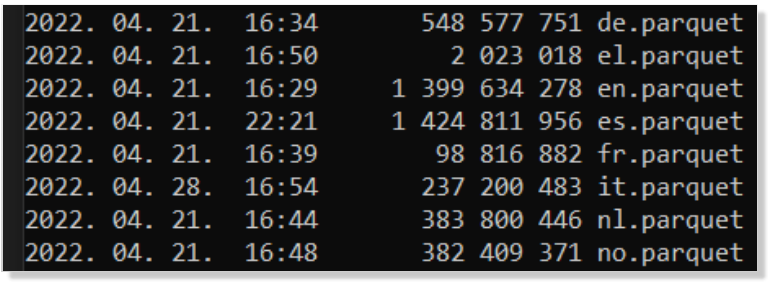


Figure 4.6: Individually pre-processed input distributed into separate input files

4.4 Dataset Splitting

Dataset Splitting is creating the Train, Test, and Validation data sets used for the training and testing of the model. The user needs to define what ratios to use for splitting. The overall goal is to create a balanced dataset, which can be challenging for the following reasons.

1. The languages need to be balanced
2. Different Record types need to be balanced for each language
3. The selected sequences can not contain repetitive addresses
4. The selected addresses can not occur in the Train and Test sets at the same time
5. The use of Synthetic data needs to be decided
6. The selected sequences need to be shuffled
7. No answer is given regarding what would be a suitable size for a training set?

These points are the results of conclusions and experiments.
1. This means that all the languages should be present with the maximum amount of available data, possibly with similar amounts of data per language. Since teaching a model for English addresses will also make the model able to handle addresses from other languages, if possible most of the data should be used for training, but more importantly, all the

target languages should be present in the training set with a minimal amount of data.
2. As was discussed earlier, the sequences were classified into three groups based on record types. It must be considered that the most miniature set of record types will be the set of SME records. From these types of sequences, it is needed to collect and keep as much as possible. SME records consist of organic and synthetic subsets.
3. Since the script is splitting complete web pages into sequences, these sequences will be linked with the same address. The conclusion is that the script needs to remove sequence duplicates based on their address.
4. If an address appears twice, once in the train and once in the test set, the model will learn the address and will also find it with possibly high accuracy in the test set, making the metrics untruthful. 5. The decision needs to be made regarding whether synthetic data needs to be used for the model. As a result of analyzing the numbers of sequences given for each record type, Synthetic SME records are being used, but not Synthetic COMPONENT or OTHER records.
6. Since the selected records need to be split into three sets, the script has to shuffle the dataset before this operation.
7. The size of the dataset will be determined by the available size of the **Original SME set**. The number of other record types (non-SME records) will be determined based on what percentage of the whole dataset the user wants the Original SME set to be selected (see figure 4.7 below). **Please note**, the numbers of TRUE_SME sequences are the number of distinct records. It is also important to note that the number of language records that need to be collected in each SYN_SME and OTHER record is determined based on the number of TRUE_SME records.

TRUE_SME RECORDS		SYN_SME RECORDS		OTHER RECORDS		*TOTALS*
en	4979	en	1245	en	6224	*12448*
nl	4829	nl	1207	nl	6036	*12073*
de	4539	de	1135	de	5674	*11348*
it	3365	it	841	it	4206	*8413*
es	3161	es	790	es	3951	*7903*
no	741	no	185	no	926	*1853*
fr	200	fr	50	fr	250	*500*
el	1	el	0	el	1	2
	21815		*5453,75*		*27268,8*	54 538
TRUE_SME	0,4	SYN_SME	0,1	OTHER	0,5	1

Figure 4.7: Example for a 40% - 10% - 50% distribution where 21 815 TRUE SME records are available

4.4.1 Dataset Splitting Procedure

In this subsection, the major steps are discussed in the *dataset_ generator.py* script, which will generate the three subsets from the pre-processed dataset.

1. Collecting TRUE SME records: For each language input file, collect all the available original records containing an address.
2. Dropping duplicates & shuffling the data.
3. Calculating the required number of sequences from **Synthetic SME** and **Other** records based on the size of the TRUE SME set.
4. Collecting SYN SME and OTHER sequences.
   (a) Dropping duplicates & shuffling the data.
   (b) Remove records where address can be already found in TRUE SME records.
   (c) Sample the required amount of records.
5. Splitting the dataset into Train, Test, and Validation sets.
   (a) Shuffling the three collected sets. (TRUE SME, SYN SME, OTHER)
   (b) Sampling and distributing the three subsets, resulting in nine sets.
   (c) Creating the three output files by merging the nine subsets.
6. Exporting the subsets into three parquet files.

The Output is 3 Parquet files ready to be used to train a model (train.parquet, test.parquet, validation.parquet).

Chapter 5

Token Classification with Machine Learning

The primary aim of the entire project is to check the feasibility of a Machine Learning solution for Address Extraction in a multilingual environment. Aside from some basic definitions, this section will discuss what tools and techniques were used for the conducted experiments and how the training was done.

5.1 Metrics

5.1.1 Classical Metrics for Classification

Some of the most widely used metrics for classification problems are Accuracy, Precision, Recall, and F1 score. These are calculated on the model with the evaluation dataset for each epoch during training.

Accuracy can be defined as the ratio of the number of correctly classified entities to the total number of entities. True Positives (TP) plus True Negatives (TN) divided by all the samples.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Recall shows the quality of a positive prediction made by the model.

$$Recall = \frac{TP}{TP + FN}$$

Precision refers to the number of true positives divided by the number of positive predictions.

$$Precision = \frac{TP}{TP + FP}$$

The **F1 score** is a performance metric for classification and is calculated as the harmonic mean of precision and recall:

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall} = \frac{2 * TP}{2 * TP + FP + FN}$$

5.1.2 Metrics for Token Classification

The indicators mentioned above are used in token classification as well; however, to compute them, the SeqEval library [Nakayama, 2018] was used. It is a Python framework for sequence labeling evaluation (The Hugging Face version was used). While the library can return overall metrics regarding all the labels, it can also produce indicator values for each entity group. For address extraction, only the Address (ADD) entity group is present; however, the COMPONENT labels can also be defined as a separate entity, resulting in individual results for labels belonging to that class. The evaluation of a sequence is visualized below with short sequences.

Option 1: Based on BILOU labeling directives, "O" is ignored from the evaluation's point of view.

```
y_true = [['O', 'O', 'O',     'B-ADD', 'I-ADD', 'L-ADD', 'O'],
['B-ADD', 'I-ADD', 'O']]
y_pred = [['O', 'O', 'B-ADD', 'I-ADD', 'I-ADD', 'L-ADD', 'O'],
['B-ADD', 'I-ADD', 'O']]

Classification Report:
              precision    recall  f1-score   support
         ADD       0.50      0.50      0.50         2
   micro avg       0.50      0.50      0.50         2
```

Option 2: The COMPONENT token is determined as a label group that must be defined. In this case, COMPONENT tokens are also taken into consideration during evaluation.

```
y_true = [['O',     'U-COM', 'B-ADD', 'B-ADD', 'I-ADD', 'L-ADD', 'O']]
y_pred = [['U-COM', 'U-COM', 'B-ADD', 'B-ADD', 'I-ADD', 'L-ADD', 'O']]
Classification Report:
              precision    recall  f1-score   support
         ADD       1.00      1.00      1.00         1
         COM       0.50      1.00      0.67         1
   micro avg       0.67      1.00      0.80         2
```

The values are defined by the SeqEval Documentation as shown below:
Accuracy = Number of elements predicted / Total number of elements
Precision = Number of predicted correct entities / Total number of predicted entities
Recall rate = Number of predicted correct entities / Total number of labeled entities

5.2 Introduction to BERT

5.2.1 What is BERT?

BERT is a model which was proposed in "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" [Devlin et al., 2018]. It is a bidirectional transformer that was trained on a huge corpus that included the Toronto Book Corpus and Wikipedia. "BERT is designed to pre-train deep bidirectional representations from the unlabeled text by jointly conditioning on both left and right context in all layers". [Huggingface, 2022]. BERT was trained with the masked language modeling (MLM) and next sentence prediction (NSP) objectives. It is efficient at predicting masked tokens in general but is not optimal for text generation.

Layers of Bert

The Base variant of the BERT model uses 12 layers of transformers block with a hidden size of 768 and has around 110M trainable parameters. On top, a classifier layer is given, which is the only layer that needs to be trained upon fine-tuning. This layer will have as many features as the number of classes the model is fine-tuned, for classification (a linear layer on top of the hidden-states output).

5.2.2 Cross Entropy

"Cross entropy is a concept used in machine learning when algorithms are created to predict from the model. The construction of the model is based on a comparison of actual and expected results." [Pytorch, 2022b]

$$-\sum_{c=1}^{M} y_{o,c} \log(p_{o,c})$$

Where:

- M - number of classes
- log - the natural log
- y - binary indicator (0|1) if class label **c** is the correct classification for observation **o**
- p - predicted probability observation **o** is of class **c**

5.2.3 Example 1: The difference between a base and a multilingual BERT model

Given good examples, the different behaviors can be well seen. In the example below, the non-multilingual model labels many words with START, MIDDLE, END, or COMPONENT tokens, unlike the multilingual which knows better which words to look for. For this, the reason is that the base model is not aware of the language differences. It lacks vocabulary, so regardless of the language the text is written in, it labels each token based on all the learned languages, which are the ones, the classifier layer that was trained (the languages which were used for fine-tuning).

Please note, that in the example below, - in the case of the non-multilingual model - the labels are entirely incorrect; thus, since a multilingual variant of BERT was used for the project, this example does not show the behavior, the project wants to achieve.

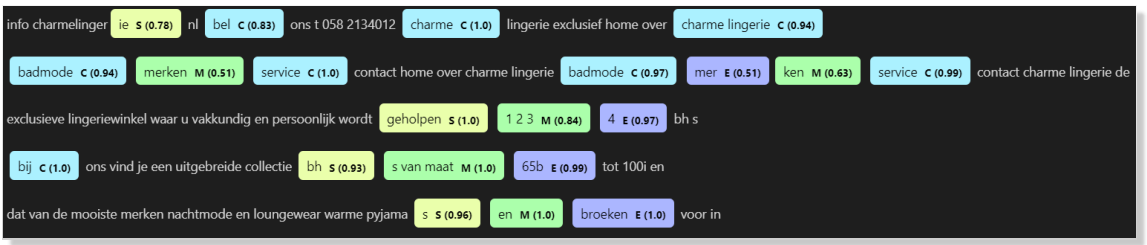


Figure 5.1: Example results, using a non-multilingual Base BERT model

The following example showcases better behavior with a multilingual model, compared to the previous one. It can be observed that the model tries to find sequences consisting of the desired order of labels (START followed by MIDDLE labels and MIDDLE followed by END); however, in this example, it fails to do so.

Figure 5.2: Example results, using a multilingual Base BERT model (mBERT)

Please note that the example above does not **contain an address**, and since none of the tokens are labeled to form a correct "accepted address" (tokens labeled in the following order: S, M, E) these tokens do not show any correct address formation and as a result, will not be extracted by the final pipeline.

5.2.4 Example 2: Location-based differences

While the model pays attention to what language the analyzed text was written in, it could also be observed in the following example that **in the case of a German address inside an English text**, if the country is added to the text, the word "Germany" will be labeled as a possible COMPONENT, while if the word "England" is appended, the model will label it with OTHER labels.

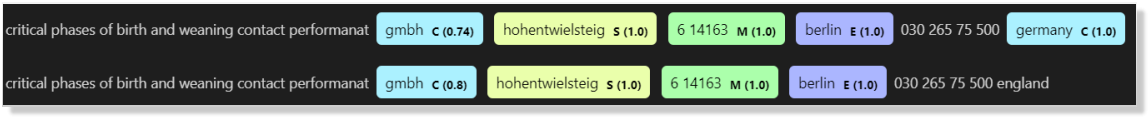


Figure 5.3: Example of a German address inside English text

5.2.5 Example 3: Capabilities of a multilingual model

Interestingly once a language was learned during pre-training, it is unnecessary to have the same language used for fine-tuning. Since the model already has some vocabulary in Italian, once fine-tuned on the given downstream task (token classification for address elements), it could even recognize Italian addresses without being fine-tuned on Italian sequences. It is essential to mention that despite this behavior, the model should be fine-tuned on all the target languages since this is just a corner case, and it might not be the same with other languages.

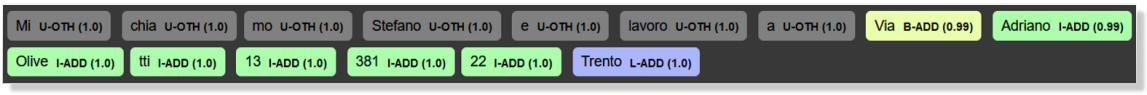


Figure 5.4: Italian address recognized without being trained on Italian sequences

5.3 Training

The training happened with the help of the Hugging Face Transformers API (Section 2.3.2). For initial training, Google Colab was used. Nevertheless, training was done on AWS EC2 instances for more extensive data sets.

5.3.1 Implementation

BERT Tokenizer

As a requirement to use the pre-trained BERT model by Hugging Face, texts need to be converted to token IDs before the model can make sense of them. This step was done during pre-processing; however additional model-dependent tokenization must be done on the sequences, which will happen with the BERT Tokenizer. The tokenizer adds special tokens used by the model: [CLS] at the beginning of the sequences and [SEP] at the end. These token tokens get the value of -100 since this value is ignored in the loss function (**Section 5.2.2. Cross entropy**). The process also splits words into smaller ones if the model does not know the given word by default (e.g., "Mi chiamo Gábor" turned into "Mi chia ##mo Gábor" which now is one token longer than earlier). This symptom occurs if the model was not pre-trained on the word ("chiamo" in this example), so it is not in its vocabulary.

Token Alignment

Since tokenization might cause the length of the sequence to change, each label's token had to be aligned with its original token. In the example below, given the example sentence, "My name is Gábor, and I live in Via Della Malpensada 90 Trento", it can be observed how the sequence is modified.

Mi	chiamo	Gábor	é	vivo	in	Via	Della	Malpensada	90	Trento
O	O	O	O	O	O	B-ADD	I-ADD	I-ADD	I-ADD	L-ADD

Figure 5.5: The mentioned example input in Italian

Sequence	[CLS]	Mi	chi	##amo	Gábor	é	vivo	in	via	Della	Mal	##pen	##sada	90	Trento	[SEP]
ID of Tokens	None	0	1	1	2	3	4	5	6	7	8	8	8	9	10	None
Label	None	O	O	O	O	O	O	O	B-ADD	I-ADD	I-ADD	I-ADD	I-ADD	I-ADD	L-ADD	None
Label ID	-100	0	0	0	0	0	0	0	1	2	2	2	2	2	3	-100

Figure 5.6: The tokenized and aligned values of the same sequence

After tokenization and additional alignment, the figure shows how the *ID of tokens* remained the same for the words, even after being split. I-ADD Labels also got added to the split address elements (*##pen* and *##sada*). Label IDs are assigned based on the label. Number 0 stands for OTHER tokens marked with an "O", while -100 marks the tokens which need to be ignored upon evaluation.

Training Loop

A pre-trained mBERT model (*bert-base-multilingual-uncased*) is imported from Hugging Face. This model cannot be used for any tasks, and it needs to be trained on a downstream task to train the extra classifier layer on top of the model.
A pre-trained model only has a classification layer that needs to be trained. This solution comes with huge advantages, including reduced training time. A training loop was created to do the training of the PyTorch model. The loop iterates over the data loader of the train set, making the forward pass through the model, then the backward pass and the optimizer step. Like Adam, an AdamW optimizer was used with a fix in how weight decay is applied. Adam optimizer [Pytorch, 2022a] involves a combination of two gradient descent methodologies (momentum and Root Mean Square Propagation):
After each epoch, the model is evaluated on the train set using the SeqEval Library (**Section 5.1.2 Metrics for token Classification**). Finally, the model is exported, which is now ready to be used for the specific task of Token Classification.

5.3.2 Training Devices

Initial training was done on Google Colab. However, using a more extensive dataset (approximately 220.000 records) increases the training time. Unfortunately, Colab only allows a user to use a single GPU for 3-4 hours. For such large datasets, an Amazon EC2 Instance was used to train the models. A **tmux** process was created so that the terminal could be closed without any problem.

```
(pytorch_p38) gabor@ip-172-31-12-183:~/training/input$ tmux new -s training
[detached (from session training)]
```

Figure 5.7: Creating a new TMUX Process

```
(pytorch_p38) gabor@ip-172-31-12-183:~/training/input$ tmux ls
training: 1 windows (created Wed May  4 09:05:57 2022) [120x27]
```

Figure 5.8: Detaching from the process

Google's Colab service provides an NVIDIA Tesla K80 or an NVIDIA Tesla T4 GPU. For ten epochs on a smaller dataset (using a Tesla K80 GPU), the training took approximately 3,5 hours (see figure below); however, since the model is only being fine-tuned, the model was only trained on the downstream task for five epochs. For the large mentioned dataset, training took around: 7-11 hours.

```
gabor@ip-172-31-12-183: ~/training/input
(pytorch_p38) gabor@ip-172-31-12-183:~/training$ python training_loop.py
[nltk_data] Downloading package punkt to /home/gabor/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
GPU is available:  True
Using custom data configuration default-dad863a67b9a83ea
Reusing dataset parquet (/home/gabor/.cache/huggingface/datasets/parquet/default-dad863a67b9a83ea/0.0.0/0b6d5799bb726b24
ad7fc7be720c170d8e497f575d02d47537de9a5bac074901)
100%|                                                                                | 2/2 [00:00<00:00, 715.14it/s]
100%|                                                                | 87260/87260 [00:11<00:00, 7640.40ex/s]
100%|                                                                | 21815/21815 [00:02<00:00, 7832.63ex/s]
100%|                                                                          | 88/88 [00:08<00:00, 10.93ba/s]
100%|                                                                          | 22/22 [00:02<00:00, 10.87ba/s]
Downloading: 100%|                                                          | 641M/641M [00:06<00:00, 106MB/s]
Some weights of the model checkpoint at bert-base-multilingual-uncased were not used when initializing BertForTokenClass
ification: ['cls.predictions.decoder.weight', 'cls.predictions.transform.LayerNorm.weight', 'cls.seq_relationship.bias',
 'cls.predictions.transform.dense.weight', 'cls.predictions.transform.LayerNorm.bias', 'cls.seq_relationship.weight', 'c
ls.predictions.bias', 'cls.predictions.transform.dense.bias']
- This IS expected if you are initializing BertForTokenClassification from the checkpoint of a model trained on another
task or with another architecture (e.g. initializing a BertForSequenceClassification model from a BertForPreTraining mod
el).
- This IS NOT expected if you are initializing BertForTokenClassification from the checkpoint of a model that you expect
 to be exactly identical (initializing a BertForSequenceClassification model from a BertForSequenceClassification model)
.
Some weights of BertForTokenClassification were not initialized from the model checkpoint at bert-base-multilingual-unca
sed and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
  1%|                                                                          | 889/109080 [02:11<3:20:46,  8.98it/s]
[training]0:python*                                                    "ip-172-31-12-183" 09:22 04-May-22
```

Figure 5.9: Model training on an Amazon EC2 Instance

5.4 Full Extractor Pipeline

The last step in development is the creation of the actual prototype. In order to use the fine-tuned model for extraction, a solution is needed for a tool to understand and extract the model's output. For this purpose, a configurable extractor was created.

5.4.1 Model Output

With the declaration of a pipeline object from the Hugging Face's transformers package, predictions could be made using the model.

```
Input: "Mi chiamo Gábor e lavoro a Via Adriano Olivetti 13 38112 Trento"
Output:
{'end': 24, 'entity': 'o', 'index': 6, 'score': 0.9996049,
'start': 18,  'word': 'lavoro'},
{'end': 26, 'entity': 'o', 'index': 7, 'score': 0.9998952,
'start': 25, 'word': 'a'},
{'end': 30, 'entity': 'B-ADD', 'index': 8, 'score': 0.9845517,
'start': 27, 'word': 'via'},
{'end': 38, 'entity': 'I-ADD', 'index': 9, 'score': 0.99656916, '
start': 31, 'word': 'adriano'}...
```

This output is then used for Visualization and Address extraction. It contains each token's beginning and ending index, including the label, its score, and the token itself.

5.4.2 Visualization

In order to make analyzing easier, dispaCy [Agata Sasiuk, 2022] was used, what is a modern syntactic dependency visualizer which helps to visualize the structure of a sentence. **Please note**, that this is not an extracted address; while the format is correct, and it will be selected for extraction, the tokens are still split, "Olive | tti" needs to be merged into "Olivetti" and the complete string needs to be extracted.

Figure 5.10: Example of a visualized sentence using a model's output

5.4.3 Result Aggregation & Thresholds

Split tokens which belong together need to be aggregated (e.g., *Mal, ##pen, ##sada*), so a personalized aggregator function was created, and the entity labels (the output of the model) were used separately. This way, a **threshold could be easily set for each label type** and it could be manually decided which token to include in the extractor function as a next step.
In case automatic aggregation would be used, the entity groups would be lost and the default class would decide what elements to extract or remove. Creating a custom aggregator allows us to create a custom pipeline and extract specific elements from the output.

5.4.4 Address extraction with RegEx

Address components can occur anywhere in the text, yet only allowed structures have to be extracted. The output is parsed with RegEx to enable the user to set the "strictness" of the address extraction. This way, different accepted patterns can be defined. The algorithm checks for the given pattern within the labels of the extracted data, where the label's score is above the set threshold.

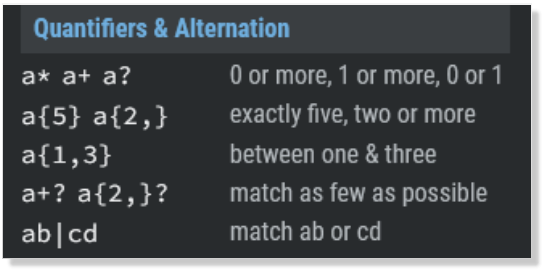


Figure 5.11: Regex Cheat sheet

The following pre-defined patterns are available by default:

1. **Level**: Strictly B, I, L addresses, where exactly one BEGIN, at least one MIDDLE, and precisely one END token is given.
   - **RegEx:** B{1}, I+, L{1}
2. **Level**: Strictly B, I, L, U addresses, using **1** tolerance level, meaning that the parser is still looking for matches identical to level 1, yet 1 COMPONENT token is allowed to appear between or outside the B, I, L elements.
   - **RegEx:** C?, B{1}, C?, I+, C?, L{1}, C?
3. **Level**: Tolerance level is increased to **2**. With this level, even 2 component elements can be accepted in between or outside the B, I, L elements.
   - **RegEx:** C{0,2}, B{1}, C{0,2}, I+, C{0,2}, L{1}, C{0,2}
4. **Level**: Multiple B and L tokens are accepted. Above level 3, the accepted patterns become unconventional. Since the model might miss-classify some tokens, the user can decide if he/she wants to allow multiple BEGINNING or LAST tokens to be extracted. More importantly, this method helps to prevent the very **first and last word** of the addresses being cut in half (loosing the first and last possible words of an address).
   - **RegEx:** C{0,2} B{1,} C{0,2} I{1,} C{0,2} L{1,} C{0,2}
5. **Level**: Multiple B, L, and C elements are also allowed. This mode was used for the experiments and measurements listed in this document.
   - **RegEx:** C{0,} B{1,} C{0,} I{1,} C{0,} L{1,} C{0,}
6. **Level**: Extract All the labeled tokens. This level was created to help human decision-making and debugging.
   - **Extracted:** Everything that got labelled as B, I, L or C

5.4.5 Fast Mode & N-grams

Upon processing whole web pages, a problem arises regarding splitting the text. In order to overcome this issue, the prototype offers two options for splitting. The model is capable of labeling 20-30 long token sequences, whereas a web page might contain thousands.

- **Naive/Fast mode:** Using this method, the input will be split into 20 long token sequences, which makes it fast and efficient. Nevertheless, there is a huge risk of the address being split into two sequences, resulting in 2 partial addresses (one in the end/one at the beginning of a sequence) not being recognized properly, causing issues.
- **N-grams:** Disabling the fast mode, the script takes another approach. N-grams are generated from the text, as shown in the figure below. There is no possible way to miss any address that was cut in half.

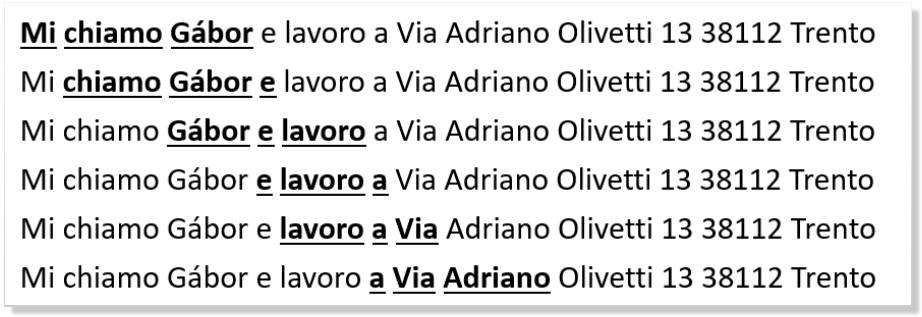


Figure 5.12: N-grams generated from the example sentence using N=3

5.4.6 Removal of Repetitive addresses

In the case of the Naive mode, there can be similar addresses in the text, while using the slower mode, due to the overlapping in the case of n-grams, the script will extract duplicates that need to be removed.
Examples: ["*Via San Pietro A Majella Vi*", "*Via San Pietro A*", "*Via Spietro A Majella 5 80138 Napoli*", "*Via San Pietro A Majella*"]

Chapter 6

Experiments

In this chapter, experiments and achieved results will be explained. Different dataset compositions were used, and different attempts were made to increase the model's performance.

6.1 Datasets

6.1.1 I. TRUE SME Dataset

Attempt:
For the very first attempt, the dataset [Train (70%), Test (20%), Validation (10%)] was selected to consist only of TRUE SME records (record type). These are the records that were **organically** containing an address from the input text. This first iteration aimed to see what the model could do with such dense data.
Result:
The model expected every sequence (split of a web page) to contain an address. As a result, it tried to label as many possible tokens as address parts as possible, which is not the desired behavior since a website usually contains only a few addresses (see figure 5.1)
Conclusion:
Additional non-address type (non-SME == OTHER) Sequences need to be added to the model, so it will attempt to label tokens as address entities, less often.

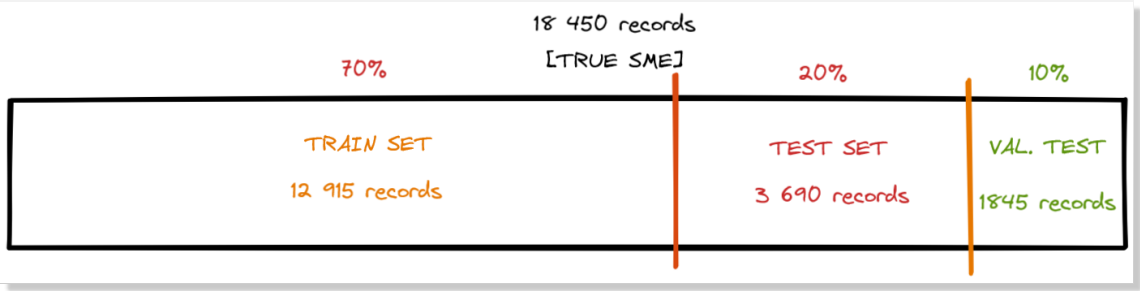


Figure 6.1: Distribution of the dataset, only containing Organic SME data records

6.1.2 II. TRUE SME + O Dataset

Attempt:

In the next iteration, the same amount of **OTHER** type sequences was added to the Train and Test splits in English, resulting in a 50%-50% distribution.

Result:

As expected, fewer tokens were labeled as address components, and the results were correctly labeled most of the time. The model's F1 score turned out to be **79,5%**; however, the validation loss was high with an average of **0.34** on the last epoch.

Conclusion:

The dataset got improved, but it needs to be balanced. The high loss had to be addressed, and a balanced dataset was needed.

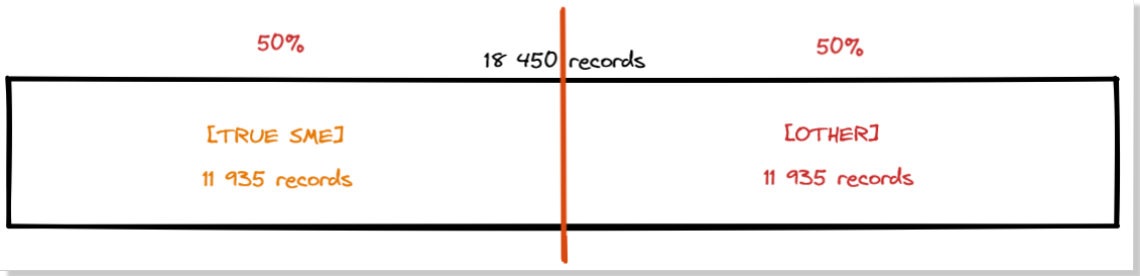


Figure 6.2: Distribution of the mentioned dataset

6.1.3 III. Dense Layers & Dropout

Attempt:

Two attempts were made to reduce overfitting. First, additional dense layers were added on top of the BERT model. Secondly, the dropout probability for all fully connected layers was increased to 0.3 and 0.5 in the embeddings, encoder, and pooler as well.

Result:

Unfortunately, the extra dense layers did not affect the indicator values; however, increasing the dropout probability to 30% reduced the numbers reasonably. Precision and F1 score: 83,2% & 83,3%). If the probability was set up to 0.5, a decrease of 8,62% was measured in F1 score.

Conclusion:

Increasing the Dropout value from the default 0.1 affected the model too much, so the modification was **rejected**.

6.1.4 Balanced Dataset - Tested Ratios

The results presented in this chapter got achieved with the help of the dataset generator script, which was explained in **Section 4.4**.
Since Multilingual Address extraction is a topic with a relatively minor amount of information available online, experiments were conducted with the given dataset (10-11GB worth of data) to see what distribution of record types is the most suitable for the train/test stets. Different datasets were created with different compositions.

IV. Organic SME 40% - Synthetic SME: 10% - Other: 50%

Attempt:
This dataset was built on the general assumption that there is a need for all the Organic (TRUE) SME records, which could be initially found in the text during pre-processing. All of the available data had to be used for training, since only 22.000 sequences were extracted from the data. In the dataset generator script, the 22.000 records were selected to represent 40% of the whole dataset. The script computes the required amount of record types for each language and randomly collects them. As a result, approximately 5.500 synthetic SME records and 27.000 other type records were collected from the pre-processed dataset. The distribution looks as shown in Figure 4.7. and 6.3.
Result:
The predictions were correct in different languages, and after five eight of fine-tuning, the model was evaluated with an F1 score of 81,6% while the precision and recall regarding address entities resulted in: 81,„12% and 82,09%. Indicators regarding Component labels were similar: Precision: 86,85%, Recall: 79,36%, F1 score: 82,94%.
Conclusion:
The generated dataset is indeed balanced, and despite that it helped a lot to increase the model's performance, other distributions need to be tested.

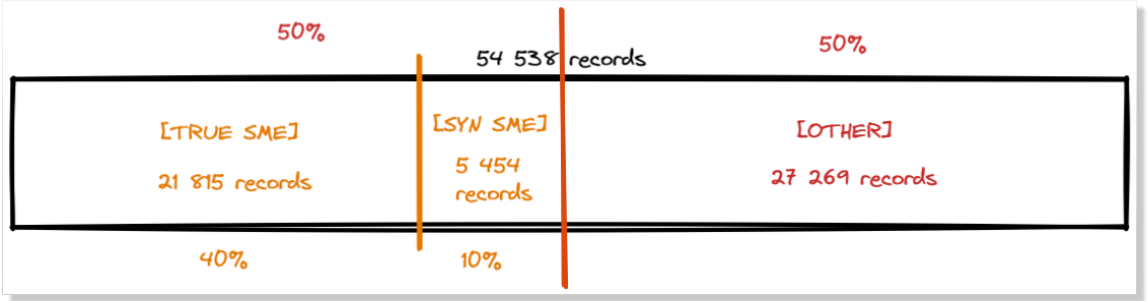


Figure 6.3: Distribution of the 40-10-50 dataset

V. Organic SME 20% - Synthetic SME: 00% - Other: 80%

Attempt:

Since a web page's textual context usually contains only a few words worth of address, a general conclusion would be that the model needs to be trained on more non-address sequences than on ones, containing an address. This dataset was constructed with the approximately 22.000 Organic SME records, being selected as 20% of the whole dataset, resulting in approximately 87.000 non-address sequences.

Result:

Address and Component precision significantly dropped to 80,4% and 83,5%. The address extractor pipeline extracted less valid information using this model, excluding some valid addresses in tested sets.

Conclusion:

While the outputs were partially incomplete/incorrect, the model performed just fine. Despite this, the conclusion is that, the tested 20-80 ratio is too extreme for the model, and more SME records are needed.

Figure 6.4: Distribution of the 20-00-80 dataset, without synthetic records

VI. Organic SME 10% - Synthetic SME: 35% - Other: 55%

Attempt:

During data pre-processing, 10% of the data was randomly selected as synthetic data, resulting in more synthetic records containing addresses than originally was given. This time, the goal was to use as many address sequences as possible.

Result:

The fine-tuning took 11 hours since 218.000 records were used, what is 2-4 times the size of the previously presented datasets.

Conclusion:

Manual testing also revealed that the model got to the level where it was confident enough to label even the most prolonged addresses, including city abbreviations (e.g., "Via Adriano Olivetti 13 38123 Trento **TN**", where **TN** was often labeled with OTHER label.

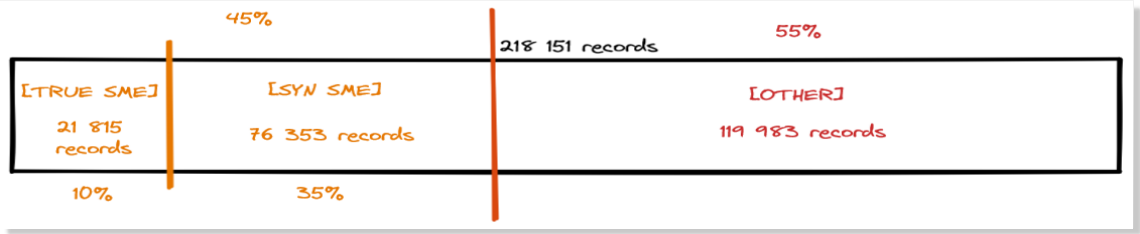


Figure 6.5: Distribution of the 10-35-55 large dataset

6.1.5 Data Augmentation

During experimentation, it was noticed that the model does not classify shorter address parts, so the augmentation of synthetic records was introduced. In order to avoid overfitting the model on the same training text/address, address duplication had to be avoided. Some of the synthetic dataset addresses were randomly truncated, showing the model that abbreviations are also possible. This augmentation was executed in 20% of the synthetic addresses where only one word was modified. In of 80% of these samples, the first letter was kept, simulating a natural abbreviation between address components. In the rest 20%, one word in each address was truncated randomly. This augmentation strongly effected the test loss, resulting in data augmentation being **removed**.
Example: *Via Garibaldi, 3, 40124 Bologna* became, *Via G 3 40124 Bologna*.

6.2 Evaluation Methods

The evaluation of the project can be done at three different points.

1. The first option is to evaluate the pre-trained and fine-tuned model, as was discussed in section **5.2. Metrics**. Using sequence evaluation, if the dataset is divided into Train and Test sets, the model can be tested for different indicators like Accuracy, Precision, Recall, or F1 score. This is a common practice to evaluate Machine Learning models. For address extraction, the extractor pipeline was introduced which uses the output of the model to decide what part of the text to grab. Unfortunately, with classic solutions the pipeline could not be tested.

2. To evaluate the recall of the entire pipeline, a custom evaluation solution also had to be created.

   (a) The addresses from the test set are looked up from the original, non-pre-processed data source. With this step, the records of the companies in the set are collected, what makes the iteration over these web pages possible. These pages are the addresses that were not used for training.

   (b) Next, to test if the known address was extracted from the web page, the script has to make sure that the original address can be found in the text. Most of the time, the address inside the text differs in format and length. A web page was said to contain the original address, if a minimum of 5 address tokens were found in the text. This threshold was set after counting how many pages in one of the test sets would contain the known address with different limits. As a result, defining the threshold below three components would result in too many unsure addresses, while increasing the number up to four or five, gave enough records to calculate a valid recall. The amount of records had to be carefully selected since for each record, address extraction had to be done for a complete web page resulting in the evaluation taking a lot of time (examples are shown for the dataset with 40-10-50 distribution).

      - Min. 3 components: 27741 records would contain an address in the test set
      - Min. 4 components: 19191 records would contain an address in the test set
      - Min. 5 components: 12555 records would contain an address in the test set
      - Min. 6 components: 8344 records would contain an address in the test set

   (c) As a final step, the extracted addresses are checked to see if any of them is the real, known address. It must be taken into account that the extracted addresses are usually different than the known address. The extracted addresses are usually shorter, and some words are different or abbreviated. Two addresses were said to be identical, if more than 80% of the address components (words/tokens) were matching. This threshold was defined based on calculations on the original dataset before pre-processing. The values are shown in the figure below. If the threshold is set to be lower than 80%, an additional 18% of the data set would be lost.

3. The above evaluation works fine in case of recall, but it raises a problem: Other metrics are hard to measure since, for a web page, only a single address is given, while the text might contain multiple addresses that are not known. Unfortunately,

since it is not possible to decide automatically if a web page exclusively contains the known address, a manually checked dataset needs to be created to test the precision of the overall solution. This dataset should strictly contain one address which would be the known address. Having this type of dataset would allow calculating many more indicator values. The main problem is that the extractor extracts multiple addresses from the text, where only one of them is undoubtedly inside the web page. In case the input text would indeed contain only one address, it could be checked if the extracted address was found or if another address was found. This result would be essential to calculate precision and other Metrics.

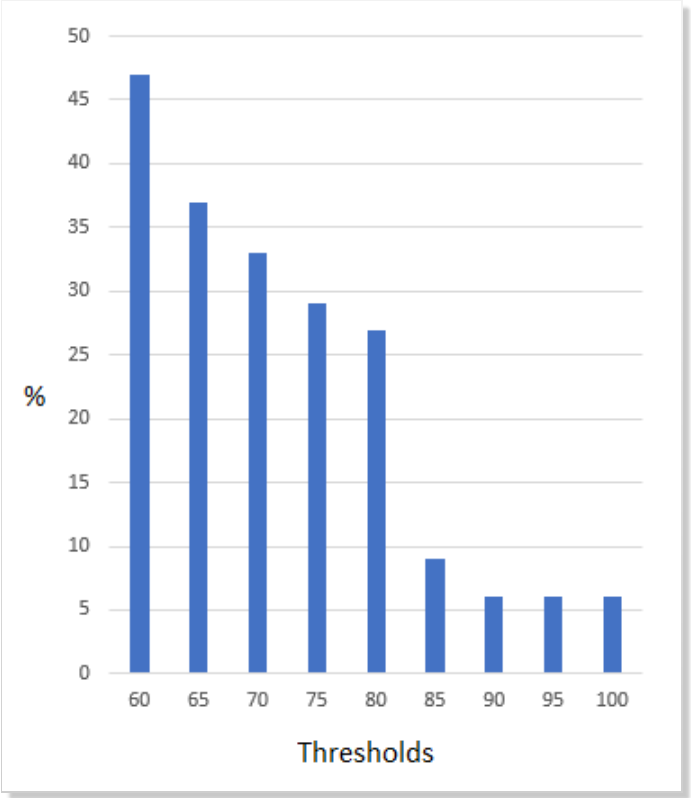


Figure 6.6: Percentages of matching addresses with each threshold

As mentioned earlier, this chart shows the data loss for each threshold separately. A significant drop can be seen when the threshold is increased above 80%. Note that 60% could also be considered a reasonable limit; however, 60% matches can not be trusted enough.

6.3 Results

6.3.1 Quantitative Results

This chapter will present some numerical results of the models which were presented.

Organic SME 40% - Synthetic SME: 10% - Other: 50% - Dropout: 0.1

Table 6.1: Metrics regarding Address and Component labels

Epoch	ADD Precision	Recall	F1 score	COM Precision	Recall	F1 score
Epoch 4	0,8170	0,8449	0,8307	0,8061	0,8917	0,8467
Epoch 5	0,8234	0,8378	0,8305	0,8437	0,8411	0,8424
Epoch 6	0,8244	0,8227	0,8235	0,8272	0,8610	0,8438
Epoch 7	0,8326	0,8375	0,8351	0,8519	0,8489	0,8504
Epoch 8	0,8112	0,8209	0,8160	0,8685	0,7936	0,8294
Epoch 16	0,8389	0,8501	0,8444	0,8440	0,8707	0,8571

The metrics are distributed into the two entity groups, Addresses (ADD) and Components (COM). For fine-tuning, 5-8 epochs are usually enough, and the results already show an increasing tendency. The best epoch seems to be Epoch 7, as Epoch 8 has already had lower indicator values and the loss also increased compared to Epoch 7.
Epoch 16 was added as proof for later comparing the models in the following subsection. At this point, Epoch 16 also seems promising in terms of performance.

Organic SME 40% - Synthetic SME: 10% - Other: 50% - Dropout: 0.3

Table 6.2: Metrics using address 0.3 dropout probability

Epoch	ADD Precision	Recall	F1 score	COM Precision	Recall	F1 score
Epoch 4	0,8067	0,8432	0,8245	0,8356	0,7702	0,8016
Epoch 5	0,8206	0,8378	0,8291	0,8507	0,7691	0,8078
Epoch 6	0,8278	0,8368	0,8322	0,8502	0,7784	0,8127
Epoch 7	0,8291	0,8427	0,8359	0,8561	0,7838	0,8183
Epoch 8	0,8321	0,8332	0,8327	0,8540	0,7703	0,8100

Comparing the results with the previous example shows that the metrics regarding address components are better, while component entities were classified less successfully.

Organic SME 20% - Synthetic SME: 0% - Other: 80%

Table 6.3: Metrics using 0.3 dropout probability

Epoch	ADD Precision	Recall	F1 score	COM Precision	Recall	F1 score
Epoch 0	0,7699	0,8238	0,7959	0,7371	0,8663	0,7965
Epoch 1	0,8036	0,8362	0,8196	0,8247	0,8313	0,8280
Epoch 2	0,8043	0,8231	0,8136	0,8346	0,8244	0,8295

Increasing the number of non-address component sequences did not help as expected. The numbers significantly dropped. The overall performance of the pipeline needs to be tested to get a more clear picture of the performance.

Organic SME 55% - Synthetic SME: 35% - Other: 55%

Table 6.4: Metrics regarding Address and Component labels

Epoch	ADD Precision	Recall	F1 score	COM Precision	Recall	F1 score
Epoch 0	0,8852	0,9175	0,9011	0,8376	0,8558	0,8466
Epoch 1	0,9060	0,9260	0,9159	0,8369	0,9183	0,8757
Epoch 2	0,9130	0,9334	0,9231	0,8892	0,8918	0,8905
Epoch 3	0,9191	0,9302	0,9246	0,9009	0,8834	0,8921
Epoch 4	0,9199	0,9307	**0,9253**	0,9004	0,8859	**0,8931**

The model's F1 score showed 92,53% on address entities and 89,31% on component entities. These values show approximately 10% improvement compared to previous iterations. Increasing the training set seems to dramatically increase the model's performance, while the training and test losses also show a nice curve.

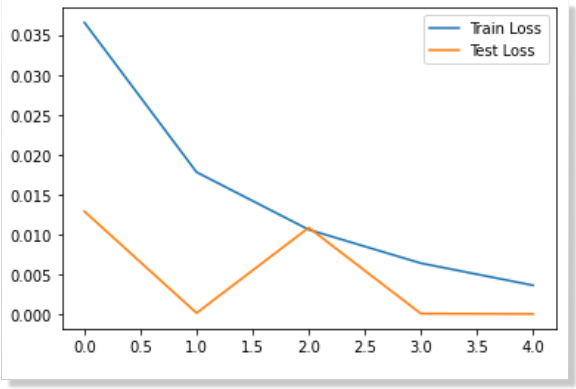


Figure 6.7: Train and Test Losses on the 10-35-55 distribution

Accuracy on Multilingual data

The following numbers show the performance of the dataset with the following composition: Organic SME 40% - Synthetic SME: 10% - Other: 50% - Dropout: 0.1 The most important information this paper needs to extract is how accurate the solution is on multilingual data. Below, the results can be seen for each language used for training. The results represent the performance of the model trained on the 40-10-50 dataset.

Table 6.5: Metrics on the Test set for each language

Language	True Predictions	False Predictions	Accuracy	% of the training set
English	1844	1163	61,32%	22,82%
Italian	2340	636	78,62%	15,42%
German	622	267	69,96%	20,8%
Norwegian	218	42	83,84%	3,39%
Spanish	3028	1681	64,3%	14,49%
French	111	128	46,44%	0,91%
Dutch	104	96	52%	22,13%
Greek	1	0	100%	0.04%

The table shown indicates how many times the address was found in the text. In the last column, the values indicate the original percentages of a given language in the training set. It can be seen that for training, there is no need for a large amount of data from a given language. Thanks to pre-training, the model already knows the " languages " themselves. A good example is Norwegian - where the address structure is very similar to the English or German one - were with only 3,39% of the training set being Norwegian, the model reached almost 84% accuracy on Norwegian addresses.

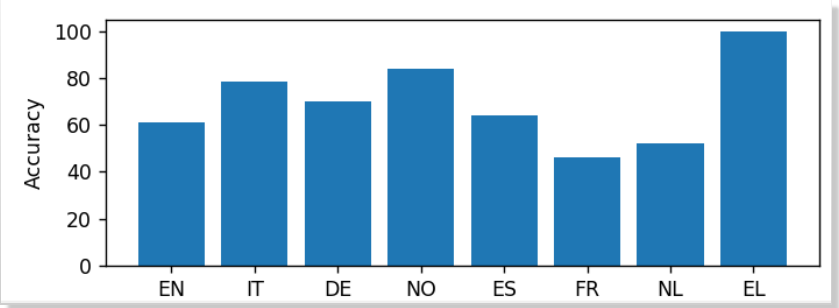


Figure 6.8: Accuracy for each Language

Pipeline Evaluation

The following table lists the mentioned models, showing which dataset based on which dataset was used. Four models are presented with two metrics listed. The pipeline recall was calculated with the method described in section **6.2. Evaluation Methods** and the "average number of extracted addresses" is self-explanatory. This second indicator needs to be lower to gain higher importance since the aim is to get as exact addresses as possible. In the case of the first model, the seventh and eighth epochs are presented to showcase the difference and improvement between two consecutive epochs. The difference in the recall is almost 1%. In the third row of the table, Epoch 16 is presented, proving that this improvement is not constant despite the better metrics of the model. The most significant change is the drastic increase of the average addresses extracted upon increasing the dropout probability. The next model was created to mimic the natural composition of a web page, where most of the content is not address-related. This approach resulted in a less confident model, resulting in a lower recall and fewer addresses found. The last model was trained on 2-4 times the amount of data as the previous ones to see if it helped or not. It can be seen that the recall is dramatically lower than in the case of other models.

Table 6.6: Scores of the Entire Pipeline using custom made evaluation

Model	Epoch	Pipeline Recall	Average Addresses Extracted
40-10-50 (Dropout: 0.1)	Epoch 7	0.6577	4.19
40-10-50 (Dropout: 0.1)	**Epoch 8**	**0.6657**	**4.08**
40-10-50 (Dropout: 0.1)	**Epoch 16**	0.6585	4.25
40-10-50 (Dropout: **0.3**)	Epoch 8	0.6325	6.29
20-00-80 (Dropout: 0.1)	Epoch 8	0.5790	3.42
10-35-55 (Dropout: 0.1)	Epoch 8	0.4732	4,81

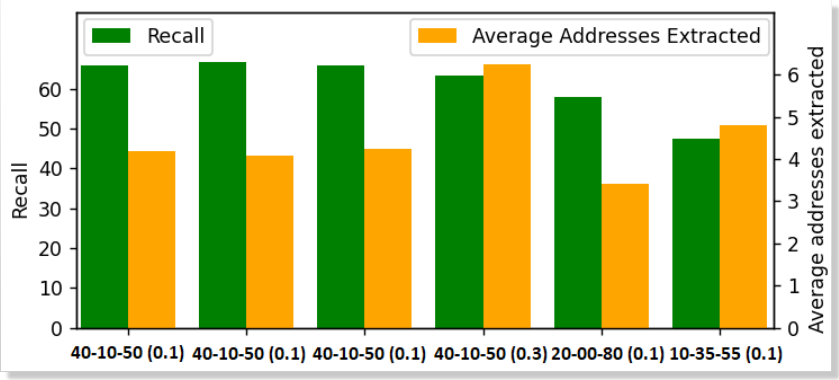


Figure 6.9: Scores of the Pipeline

6.3.2 Qualitative Results

Address Extraction using the entire pipeline

The model can achieve some spot-on labeling that was showcased in earlier examples. In this section, some real-life scenarios will be presented using the entire extractor pipeline using the model's output. The extractor can be used from code in the following way:

```
[43] extract_addresses(input_text, LEVEL, fast_mode=True)
```

Figure 6.10: Address Extraction from code, using fast_mode

The user must define the level of strictness (integer value between 1 and 7) and decide which mode to use. As mentioned earlier, the fast mode can extract addresses from a web page in 2-10 seconds, whereas, for the same input, the slower mode (using n-grams) takes 1-2 minutes. The website content of SpazioDati contains the same address twice, once on the contact page and once in the footer. The output looks like the following using the fast, naive mode.

```
100% 23/23 [00:03<00:00, 7.03it/s]
--- 3.3769848346710205 seconds => 0.05628308057785034 minutes ---
{'Via A Olivetti 13 38122 Trento',
 'Via Adriano Olivetti 13 38122 Trento Trento'}
```

Figure 6.11: Output of fast mode on SpazioDati's website.

When the fast mode is disabled, the script is expected to return more addresses since there is a possibility that an address was cut in half using the naive mode. On the other hand, using n-grams allows the model to examine the text better, resulting in extracted non-address but address-like strings. The example below is the model's output trained on the dataset constructed with the distribution of 40% Organic SME, 10% Synthetic SME, and 50% Other type records. It is clear that the extractor's output needs to be checked, and non-address elements need to be removed in the future with an address parser.

```
100% 433/433 [01:04<00:00, 6.61it/s]
--- 64.24306344985962 seconds => 1.070717724164327 minutes ---
{'13 38122 Trento Trento Rea',
 'Cerved Group Spa',
 'European Regional Development Fund',
 'Lgs D 23101',
 'Provincia Autonoma Di Trento Trentino',
 'Via Adriano Olivetti 13 38122 Trento Trento',
 'Via A Olivetti 13 38122 Trento Tn'}
```

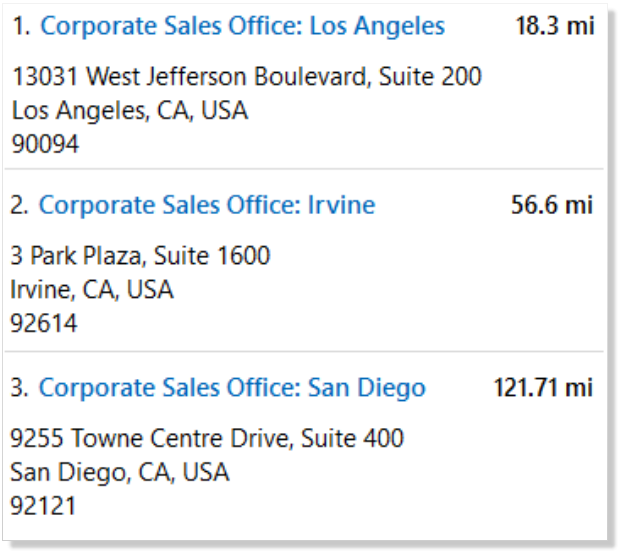
Figure 6.12: Output of N-gram mode on SpazioDati's website.

Examples from Different web pages

In this subsection, well-known company websites were tested to determine how the extractor performs in different languages. **Please note that the screenshots only show the relevant data we are expecting from the model, however the whole web page was fed to the pipeline.**

English US Addresses - Microsoft

Input URL: `https://www.microsoft.com/en-us/about/officelocator?Location=91303`



Output: {'13031 West Jefferson Boulevard Suite 200 Los Angeles Ca', '92121 View Directions Road', '9255 Towne Centre Drive Suite 400 San Diego Ca', 'Irvine 566 Mi 3 Park Plaza Suite 1600 Irvine Ca', 'Los Angeles 183 Mi 13031 West Jefferson Boulevard', 'Microsoft Tech Community'}

Italian Addresses in English Environment ("Gelato Near me" Google Search)

The pipeline extracts multiple addresses from the output. A Google search was made with "Gelato near me" (Ice cream near me) to showcase its potential. The textual content of the results was fed to the pipeline.

Input URL: `https://www.google.hu/search?q=gelato+near+me`

Output: {'Al Ponte 40336', 'Bistro Alle Torri 40124', 'Corso 3 Novembre 1918', 'Gelateria Robin Trento 43290', 'La Delizia 43155', 'La Gelateria 46153', 'Piazza Del Duomo 27', 'Shop Via 5 Vigilio 17 Near', 'Shop Viale Verona 69 Inst', 'Tro Alle Torri 40124', 'Via Carlo Esterle 7 Near', 'Via Dei Colli 2', 'Via Dei Mille 1719', 'Via Enrico Coed 76', 'Via Giuseppe Garibaldi 9', 'Via Giuseppe Grazioli 8', 'Via Giuseppe Mazzini 20 Near', 'Via Paolo Ossmazzurana 35', 'Via Paolo Ossmazzurana 41', 'Via Rodolfo Belenzani 50'}

The input did not contain zip codes or city names, and still, all the 20 results were extracted nicely. Due to the format of the result, the length of the addresses is short in the input. As a side effect, the word "Near" can be found after the actual addresses in some cases. This issue can be solved by training the model on shorter addresses.

German Addresses - Volkswagen

Input URL: `https://www.volkswagenag.com/de/meta/provider-identification.html`

Output:
{'Berliner Ring 2 38440 Wolfsburg'}

French Addresses - Total Energies

Input URL: `https://www.totalenergies.fr/mentions-legales`

Output:
{'Rue Louis Armand 75015 Paris'}

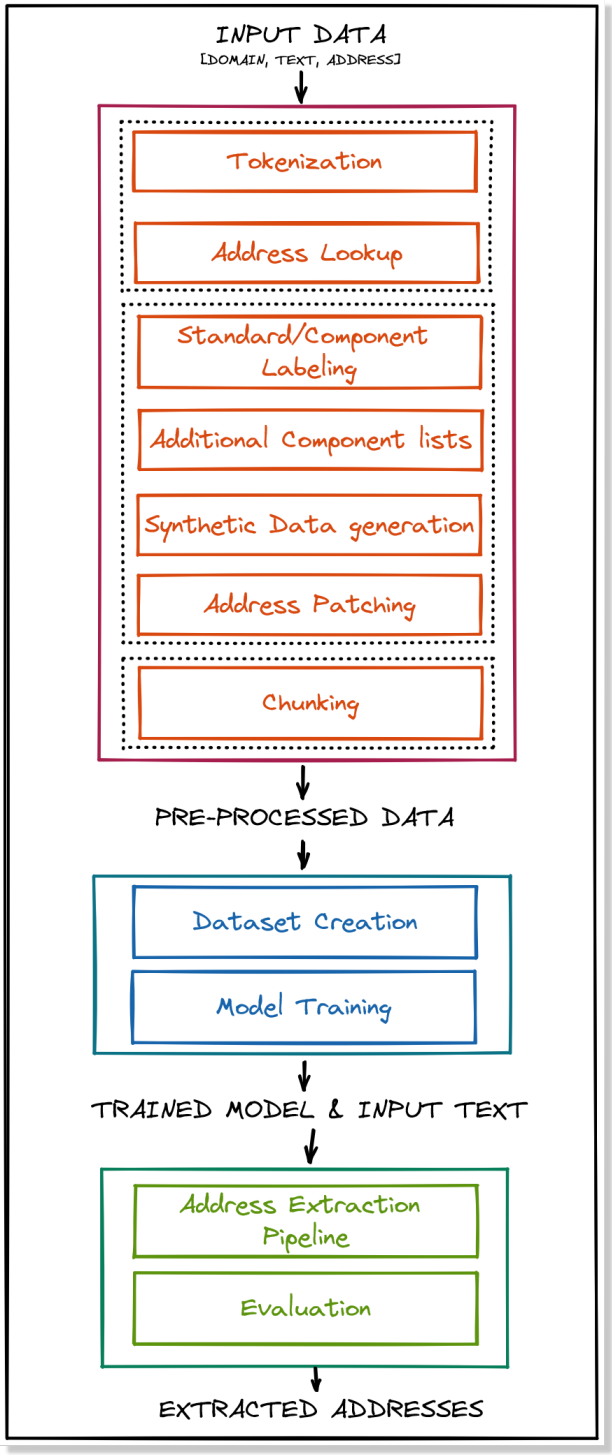
Please note, that the beginning of the address is missing, and while the extracted information is a Street level address, it is not complete.

Italian Addresses - Belka

To showcase a corner case, in the following Italian example, the address is unusually long, containing the corporation's name inside the address. This phenomenon often happens on websites; however, the training set did not cover such cases, resulting in the model not labeling the address correctly. Input URL: `https://www.belkadigital.com/contattaci`

Output:
{'Da Sanseverino 95 Co Impact Hub Trento Tn', 'Via Parma 29 10152 Torino', 'Via Parma 29 Torino To', 'Via Roberto Da Sanseverino 95'}

6.4 One Pager of the whole process



The input data contains the web domain, the text of the page, the language, and the address which belongs to the web page. This data is first pre-processed throughout multiple steps. The text is tokenized and is checked if it contains the known address. Next, the labeler labels every token with one of the START, MIDDLE, END, COMPONENT, or OTHER labels. The labeler is configurable with multiple options. It is possible to generate synthetic data for input records where the address can not be found in the text. Partial addresses can be patched, so all the tokens are labeled correctly, and as the last step, the labeled tokens are chunked into sequences that can be used to train the model. These sequences contain all the essential information when it is exported. The composition of the different record types needs to be defined. Next, the model is fine-tuned, and an mBERT model is trained. The model is trained for the downstream task of Token Classification, and the Address Extractor Pipeline analyzes the output. The pipeline requires the user to define a "strictness" of acceptance level, which decides what to extract as an "accepted address." A custom evaluation method was also created to measure the accuracy of the pipeline.

Chapter 7

Conclusions

This thesis addresses the problem of postal address extraction from textual data, coming from web pages. The difference between Address extraction and parsing was clarified. Research revealed that Machine Learning solutions improved significantly in the last decade, creating powerful models to solve the related issue of Token Classification in a multilingual environment. It was showcased that multilingual models are more effective than any other solutions used in the past.

Pre-training comes with huge benefits, offering state-of-the-art models for users. Fine-tuning a model is a robust tool, resulting in the data being the most crucial component. Such multilingual datasets are not publicly available and are hard to create. Addresses are delicate entities, so a configurable pre-processing pipeline was created. In order to find a full or a partial known address in a text address, address patching was also implemented. This pipeline can be used on any crawled data by anyone if structured correctly.

Balancing the dataset proved to be one of the most critical aspects of creating a proper model. Different dataset ratios were tested and presented, resulting in a model with reliable predictions.

In order to use and understand the output of the model, an extraction pipeline was introduced, which looks for different label patterns using Regex. The input had to be chunked in order to feed text data to the models. For text chunking, two solutions were introduced. A fast one, using a naive splitting solution carrying the risk of splitting an address into two separate sequences, and a slower but sophisticated one, using n-grams for splitting.

A custom evaluation method was created to test the whole pipeline's reliability using the trained model. Original, not pre-processed web pages that were not used to train the given model (the test set) are fed through the pipeline. Decisions had to be made to decide when to consider an address found in the text and when to consider an address successfully extracted. Based on this evaluation method and using sequence evaluation on the model

itself, a brief view was presented on how such models can be utilized and how multilingual address extraction is feasible.
Points to mention:

- From each language, it is recommended to feed the same amount of records to the model to remain balanced.
- Different ML Models can be used for the project, yet mBERT proved to be extremely flexible with Multilingual data for Token Classification.
- The dataset does not form a part of the submitted thesis; however, simple (open-source) crawlers were also created and submitted with the thesis.
- Using n-grams over a naive splitting method comes with a huge trade-off. Instead of a few seconds, extraction can last for multiple minutes, depending on the size of the input text. On the other hand, the output list is guaranteed to be more extensive than using the faster solution.
- The user has to select a specific "strictness" level upon extraction, so the extractor pipeline knows which patterns to apply to the model's output. (Default: Level 5)
- To evaluate the pipeline as a whole, a single address was defined as extracted from a text if it shows 80% similarity to the one extracted from the text (based on tokens).

7.1 Answering of the Research Questions

In this section, the proposed questions that were described in section **3.1. Research Questions to be answered** will be answered.

7.1.1 What is the state-of-the-art solution for multilingual NLP?

Multilingual Machine Learning Models! In the last years, Google released BERT, mBERT, T5, and mT5 models, while Microsoft and other tech giants also developed state-of-the-art models, which were made available in a pre-trained form to the public. These models were made for NLP tasks, like question answering. Since Microsoft did not publish its models, BERT become one of the most popular models. It can be used for multiple tasks, including token classification. Older methods like CRF [TowardsDataScienceCRF, 2022] or SVM [MediumSVM, 2022] are not capable of producing such high scores, like BERT: [Huggingface, 2022]

7.1.2 How can we use a large set of example addresses to build an Addresses Detection Model

The problem field is Token Classification. The most widely used solutions are NER and Chunking. BILOU labeling was used in this thesis, and it is one of the most trustworthy labeling solutions for chunking. With chunking, tokens can be marked if they appear in the beginning, middle, or end of the address. Furthermore, **unit** length tokens can also be utilized like COMPONENT tokens.
The dataset needs to be balanced and well distributed. Upon collecting data for a multilingual solution, it is vital to get enough data from each language. Furthermore, a single address should be used only once for training; duplicates should not be allowed. Due to the similarities between addresses, specific components will occur multiple times in the training set by default. It is important to note that sequences that do not contain addresses (just ordinary text elements) also need to be used for fine-tuning. After the data is prepared, a model can be trained for Token Classification and used for Address Detection.

7.1.3 How can we make this process efficient for millions of websites?

This question can be split into two parts. On the one hand, the pre-processing needs to be sufficient, while on the other hand, the extraction process also needs to be quick.
As millions of web pages need to be processed and multiple string operations need to be done to a single web page, the procedure must be optimized. Data is read and processed in chunks and stored in parquet files in order to save space.
To train the model efficiently, the model is trained with sequences for token classification problems. The model is trying to predict a single token based on other ones in the sequence. In conclusion, if the sequence is too long, the prediction will be slow. If the model is trained and tested on sequences, the whole process will be faster.
During prediction, another problem is the splitting of the input text. Using a naive splitting method where the text is split at every **Nth** token is fast, and is used for the final evaluation. The N-gram solution is recommended if the extractor is used on a minor test set (or a single web page), as the output will consist of more results, processing time will not be too long.

7.1.4 How will the quality differ in different European countries? Will the alphabet influence the results?

The expectations for this question would be the following: In case a language is not used during fine-tuning, the model will not be able to extract information from the given language properly. The statement is only half true. Interestingly, since the model was pre-

trained in different languages, the alphabet causes difficulties for the model. For instance, the base mBERT model was not pre-trained on Hungarian text, so the model does not have a vocabulary of Hungarian words. This situation results in many words being split into different tokens during the tokenization by the BERT tokenizer, causing problems with labeling since the model will not be able to decide over the unknown split words. On the other hand, a counterexample is the case of Italian. Since Italian has similar words and sentence structures to English, the model can find Italian addresses (address like tokens) even when the model is not fine-tuned on Italian sequences (See the earlier example at **Figure 5.3.5**).

7.2 Next Steps

Possible next steps include extending the languages, especially ones in different alphabets. After approximately 200.000 sequences, the fine-tuning seems to get to the level where it is not improving anymore. Instead, different languages with unique alphabets should be added.
Since the model can return false addresses like "European Regional Development Fund", the output addresses should be analyzed with a parser like libpostal, to decide if the extracted information is a legitimate address with each address part in the data or not.

7.3 What is missing

The prototype of an address extraction solution needs to be deployed and should be made easily accessible through an API. The trained model needs to be made available for the extractor pipeline, so a simple Text Input and Output API call can be made.

7.4 Different techniques that could be tried

Hugging Face is currently running a many-month-long project. This project is the training of the world's largest open-source multilingual language model. The training lasts approximately 124 days and is huge cooperation with many researchers worldwide. The BigScience project is training this multilingual model on 176 billion parameters, and it surely will be groundbreaking in many NLP tasks. When the model is finished and is publicly made available, it could be interesting to see how it performs after being trained on the downstream task of token classification and how it behaves when used for address extraction. [BigScience, 2022] From an experimental point of view, it could also be interesting to generate a 100% synthetic dataset and train a large mBERT model on it.

List of Figures

List of Tables

Bibliography

[Abid et al., 2018] Abid, N., Ul-Hasan, A., and Shafait, F. (2018). Deepparse: A trainable postal address parser. pages 1–8.

[Agata Sasiuk, 2022] Agata Sasiuk, K. (Accesssed: 2022). Displacy dependency visualizer. https://spacy.io/universe/project/displacy.

[BERT, 2022] BERT, G. R. (Accesssed: 2022). Google bert and mbert. https://github.com/google-research/bert.

[BigScience, 2022] BigScience (Accesssed: 2022). Bigscience. https://bigscience.huggingface.co/.

[Chang and Li, 2010] Chang, C.-H. and Li, S.-Y. (2010). Mapmarker: Extraction of postal addresses and associated information for general web pages. volume 1, pages 105–111.

[Devlin et al., 2018] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding.

[D'Souza, 2022] D'Souza, J. (Accesssed: 2022). Chunking and pos tagging. https://medium.com/greyatom/learning-pos-tagging-chunking-in-nlp-85f7f811a8cba.

[Huggingface, 2022] Huggingface (Accesssed: 2022). Bert transformer. https://huggingface.co/docs/transformers/model_doc/bert#overview.

[MediumSVM, 2022] MediumSVM (Accesssed: 2022). Support vector machines for nlp. https://medium.com/@bedigunjit/simple-guide-to-text-classification-nlp-using-svm-and-naive-bayes-with-python-421db3a72d34.

[Nakayama, 2018] Nakayama, H. (2018). seqeval: A python framework for sequence labeling evaluation. Software available from https://github.com/chakki-works/seqeval.

[Openvenues, 2018] Openvenues (2018). Libpostal: International street address nlp. Software available from https://github.com/openvenues/libpostal.

[Pytorch, 2022a] Pytorch (Accesssed: 2022a). Adam optimizer. https://pytorch.org/docs/stable/optim.html.

[Pytorch, 2022b] Pytorch (Accesssed: 2022b). Cross entropy loss. https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html.

[Ramshaw and Marcus, 1995] Ramshaw, L. A. and Marcus, M. P. (1995). Text chunking using transformation-based learning.

[Stefangabos, 2022] Stefangabos (Accesssed: 2022). World countries dataset. http://stefangabos.github.io/world_countries/#top.

[TowardsDataScience, 2022] TowardsDataScience (Accesssed: 2022). Pos tagging. https://towardsdatascience.com/part-of-speech-tagging-for-beginners-3a0754b2ebba.

[TowardsDataScienceCRF, 2022] TowardsDataScienceCRF (Accesssed: 2022). Conditional random fields. https://towardsdatascience.com/conditional-random-fields-explained-e5b8256da776.

[TowardsDataScienceNER, 2022] TowardsDataScienceNER (Accesssed: 2022). Named entity recognition. https://towardsdatascience.com/named-entity-recognition-with-bert-in-pytorch-a454405e0b6a.

[Vohra, 2016] Vohra, D. (2016). *Apache Parquet*, pages 325–335. Apress, Berkeley, CA.

[Yassine et al., 2020] Yassine, M., Beauchemin, D., Laviolette, F., and Lamontagne, L. (2020). Leveraging Subword Embeddings for Multinational Address Parsing.

[Yu, 2007] Yu, Z. (2007). *High Accuracy Postal Address Extraction From Web Pages*. Dalhousie University Halifax, Nova Scotia.